"Loose coupling is a gift your future self will thank you for."

MODULE 22

Spring Core and Inversion of Control

Build loosely coupled, maintainable, and testable enterprise applications with Spring's IoC container and dependency injection.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 22 OVERVIEW

Build loosely coupled, maintainable, and testable enterprise applications with Spring's IoC container and dependency injection.

Spring's IoC container is the foundation for everything ahead this week -- this module covers dependency injection, bean lifecycle, scopes, and stereotype annotations through an Order Management System scenario and the Northstar CRM bean-graph lab.

DURATION & LAB

4 Hours Theory

IoC container fundamentals, constructor/setter/field injection, bean lifecycle and scope, and stereotype-driven component scanning.

2 Hours Lab

Northstar CRM Bean Graph: refactor to constructor injection, add stereotypes, and document the dependency graph.

Scenario

Order Management System: replace tight coupling and manual `new` calls with an IoC-managed, testable dependency graph.

MODULE ARC

Understand IoC

Why tight coupling breaks at scale, and how the IoC container inverts object creation.

Apply Dependency Injection

Constructor, setter, and field injection; bean lifecycle, scope, and stereotype annotations.

Build the Lab

Refactor Northstar CRM to constructor DI with @Component/@Service/@Repository stereotypes.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours hands-on lab. Spring IoC and Dependency Injection turn a tangled dependency graph into a maintainable, testable, and scalable application.

WHY SPRING DOMINATES ENTERPRISE JAVA DEVELOPMENT

Spring provides everything enterprises need to build robust, scalable, secure, maintainable applications.

KEY REASONS

Productivity
Conventions and auto-config speed up delivery.

Enterprise Scale
Proven stability for high-traffic applications.

Full Ecosystem
Web, Data, Security, Messaging, Cloud, Testing.

Secure by Design
Deep integration with Spring Security.

Future-Ready
Supports microservices and cloud-native design.

Trusted Community
Huge community and wide enterprise adoption.

BUSINESS IMPACT

- **Faster development —** focus on business logic, not infrastructure.
- **Higher quality —** cleaner, maintainable, testable code.
- **Better scalability —** well-structured applications that grow with the business.

KEY TAKEAWAY Spring is not just a framework — it's a complete platform for production-ready, enterprise-grade applications.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to:

- **Understand Spring Fundamentals —** explain what Spring is, the problems it solves, and the ecosystem/architecture.
- **Explain Inversion of Control (IoC) —** describe how the IoC container manages object creation and dependencies.
- **Apply Dependency Injection —** implement and differentiate constructor, setter, and field injection.
- **Work with Spring Beans —** define beans, understand their lifecycle, and configure singleton/prototype scopes.
- **Use Component Scanning and Stereotypes —** apply @Component, @Service, @Repository, and @Controller.
- **Design Loosely Coupled Applications —** build maintainable, testable systems using IoC and bean relationships.

KEY TAKEAWAY These objectives build the core concepts and hands-on skills to build robust, maintainable enterprise applications.

ENTERPRISE SCENARIO: BUILDING MAINTAINABLE APPLICATIONS

A large-scale Order Management System must handle high traffic, evolve continuously, and integrate reliably.

BUSINESS CHALLENGES

- **Frequent Changes —** requirements shift constantly; code must adapt without breaking.
- **Tight Coupling —** makes testing, maintenance, and updates difficult and risky.
- **Complex Dependencies —** databases, external APIs, messaging, third-party libraries.
- **Large Teams —** multiple teams must work in parallel with minimal conflicts.

HOW SPRING CORE (IoC & DI) HELPS

- **Loose Coupling —** components depend on abstractions, not concrete implementations.
- **Dependency Injection —** the container manages dependencies and their lifecycle.
- **Easier Testing —** mocks and stubs are injected for unit and integration tests.
- **Scalability —** supports modular, extensible, enterprise-grade applications.

BUSINESS OUTCOMES

Faster delivery | **Fewer defects** | **Easier onboarding** | **Lower maintenance cost** | **Better resilience**

KEY TAKEAWAY Spring IoC and DI turn a tangled dependency graph into a maintainable, testable, and scalable application.

WHAT IS THE SPRING FRAMEWORK?

A comprehensive, lightweight, open-source framework for building enterprise-grade Java applications.

Spring provides a programming and configuration model that helps developers build robust, maintainable, testable applications quickly.

KEY CAPABILITIES

Core Container

Provides the IoC container and dependency injection.

AOP Support

Aspect-oriented programming for cross-cutting concerns.

Rich Ecosystem

Data access, web, security, messaging, and testing.

Enterprise Ready

Built for reliability, scalability, and performance.

WHY SPRING WAS CREATED

- Simplify complex enterprise application development.
- Promote loose coupling through Inversion of Control (IoC).
- Improve testability, maintainability, and reusability of code.
- Accelerate development and increase productivity.

KEY TAKEAWAY Spring provides the foundation for modern Java applications so you can focus on business logic, not boilerplate.

PROBLEMS SOLVED BY SPRING

Spring addresses the core complexities of enterprise Java development with proven solutions.

Common Problem	How Spring Solves It
Tight Coupling	IoC & DI — Spring manages object creation and wiring, promoting loose coupling.
Complex Object Management	IoC Container — Spring creates, configures, and manages objects automatically.
Difficult Maintenance	Centralized Configuration — externalized, annotation-based configuration improves clarity.
Poor Testability	Testable Code — loose coupling makes components easy to mock and test in isolation.
Repetitive Boilerplate	Cross-Cutting Concerns (AOP) — declarative transactions, security, caching, logging.

KEY TAKEAWAY Spring eliminates complexity and boilerplate, helping teams build robust, maintainable, scalable applications.

SPRING ECOSYSTEM OVERVIEW

A family of projects that work seamlessly together to build modern, enterprise-ready Java applications.

Core Foundation

Spring Framework, Context, SpEL — building blocks every other project uses.

Web

Spring MVC, WebFlux, Boot, Thymeleaf for web apps and REST services.

Data Access

Spring Data JPA, JDBC, MongoDB, Redis for SQL and NoSQL.

Security

Spring Security and OAuth2 for authentication and authorization.

Integration & Messaging

Spring Integration, Kafka, AMQP, JMS for connecting systems.

CLOUD & DEVOPS ENABLEMENT

Spring Boot | **Spring Cloud** | **Spring Cloud Config** | **Spring Cloud Gateway**

BENEFITS OF THE ECOSYSTEM

- Faster development of production-ready applications.
- High productivity — focus on business logic, not infrastructure.
- Large, active community and rich documentation.

KEY TAKEAWAY Spring is more than a framework — it's a complete ecosystem that accelerates enterprise Java development.

Spring Ecosystem Overview

A modular ecosystem for building modern, production-ready applications

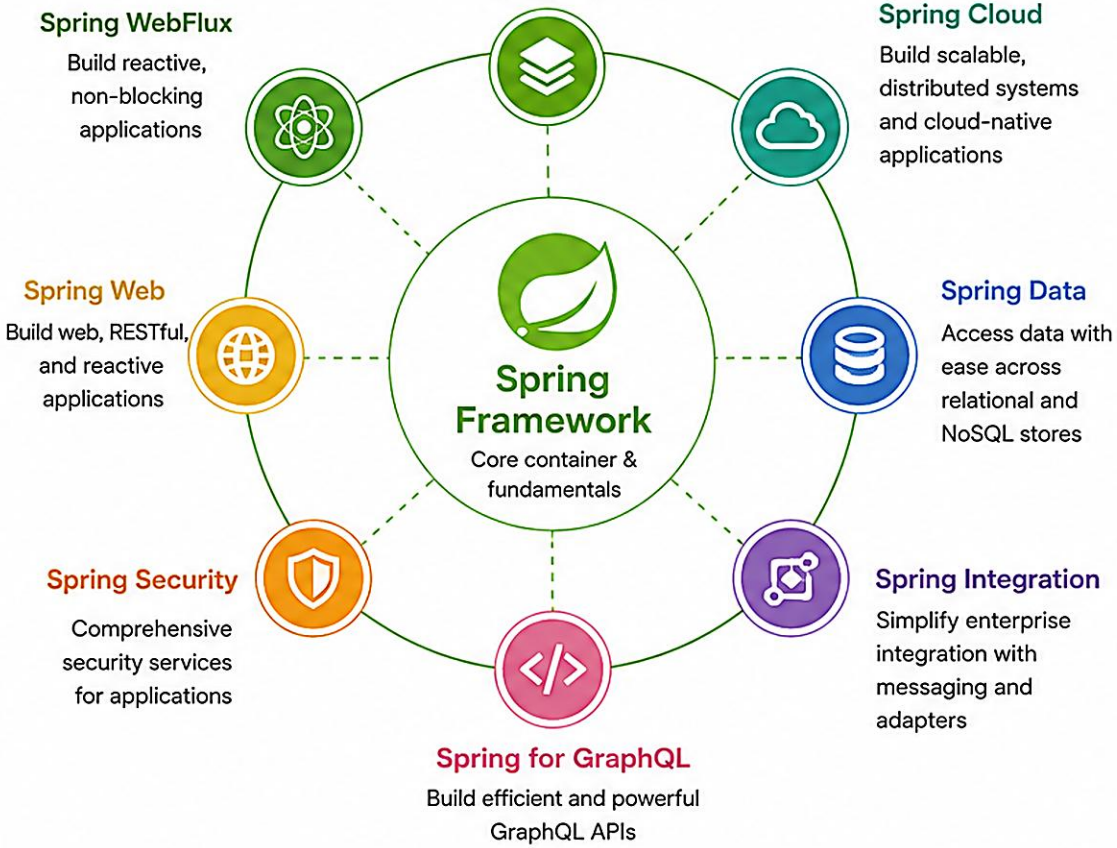
DEVELOPER EXPERIENCE

Spring Initializr
Bootstrap your Spring projects

Spring Tools 4
IDE support for faster development

Spring Boot DevTools
Live reload and enhanced developer productivity

Spring Boot CLI
Build and run Spring apps from the command line



CLOUD & INFRASTRUCTURE

Spring Cloud Kubernetes
Deploy and manage applications on Kubernetes

Spring Cloud Config
Externalized configuration for distributed systems

Spring Cloud Gateway
API gateway for routing, filtering, and security

FOUNDATION PROJECTS

Project Reactor
Reactive programming foundation

Spring Core
Core utilities and IoC container

Spring Expression Language
Powerful expression language for Spring

ONE ECOSYSTEM. ENDLESS POSSIBILLITIES. | Designed to be modular, flexible, and compatible—use what you need, when you need it.

SPRING ARCHITECTURE OVERVIEW

A layered architecture where each layer builds on the one below it, with the IoC container at the core.

Layer	Key Components	Responsibility
Application	Web UI, REST Controllers, Mobile Clients	Handles requests/responses; exposes REST APIs.
Service	@Service beans, business logic	Business logic and workflows; uses the data layer.
Data Access	@Repository beans, Spring Data JPA	Abstracts persistence; CRUD operations and queries.
Infrastructure	DataSource, Transactions, Security, Messaging	Technical capabilities shared across the app.
Core Container	IoC Container — Beans, DI, Lifecycle, Scopes	Creates, injects, configures, and manages every bean.

CROSS-CUTTING CONCERNS (APPLIED ACROSS ALL LAYERS)

Logging **Validation** **AOP** **Monitoring** **Exception Handling**

KEY TAKEAWAY Spring's layered architecture promotes maintainability, scalability, and testability through IoC and DI.

Spring Framework Architecture Overview

A layered architecture designed for modularity, flexibility, and enterprise scalability

spring by VMware Tanzu

Key Principles

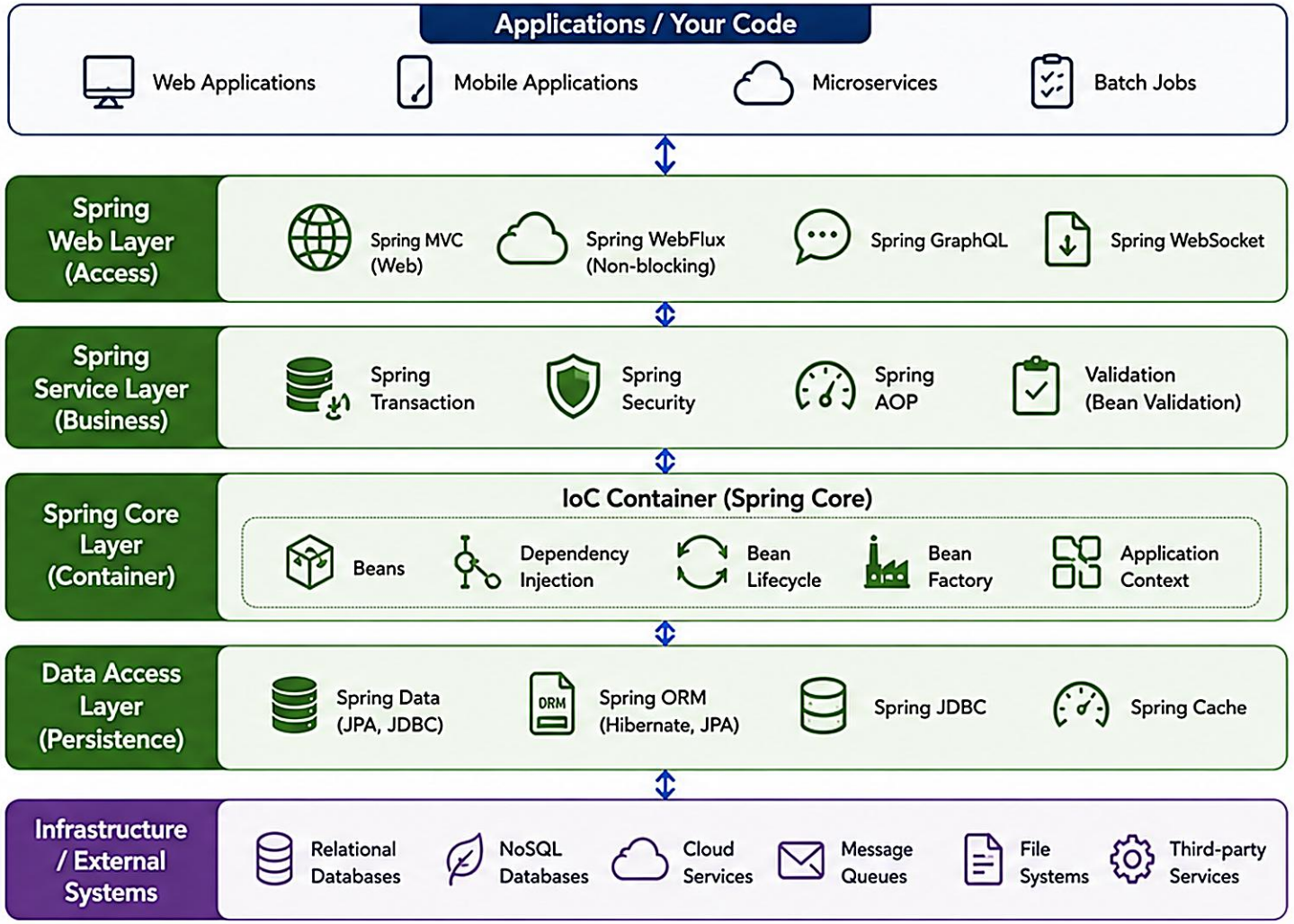
Modular
Components are modular and optional.

Loosely Coupled
Promotes separation of concerns and clean design.

Extensible
Easy to extend and integrate with other technologies.

Testable
Design supports unit and integration testing.

Enterprise Ready
Built for scalability, performance, and reliability.



Cross-Cutting Concerns

- Security
- Monitoring / Observability
- Logging
- Metrics
- Tracing
- Configuration Management

Technology Agnostic

Spring works with a wide range of technologies and adapts to your architecture.

Java · Kotlin · Groovy · Cloud Native

Core Idea: The Spring Framework provides a comprehensive programming and configuration model for modern Java applications.
It is built around the IoC container, which manages the lifecycle and dependencies of application components (beans).

UNDERSTANDING TIGHT COUPLING

Tight coupling occurs when a class depends directly on another class's concrete implementation.

OrderService.java (tightly coupled)

```
public class OrderService {
    // Direct dependency — hard-coded
    private EmailService emailService
        = new EmailService();

    public void placeOrder(Order order) {
        emailService.sendOrderConfirmation(order);
    }
}
```

WHAT IS TIGHT COUPLING?

- One class knows too much about another class's internals.
- It depends on a concrete implementation, not an abstraction.
- Any change to the dependency forces a change to the client.
- Harder to test, extend, and reuse in isolation.

Real-world analogy: a car welded to a specific engine — swapping the engine means rebuilding the entire car.

KEY TAKEAWAY Tight coupling makes code hard to change, test, and maintain — the opposite of what enterprise systems need.

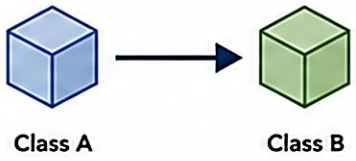
Understanding Tight Coupling

Components are strongly dependent on each other.
A change in one forces changes in many.

spring by VMware Tanzu

What is Tight Coupling?

Tight coupling occurs when one component (class/module) has direct knowledge of, or depends on, the internal implementation of another component.



Class A directly creates, uses, and relies on Class B.

Example: Order Service Directly Depends on MySQL Implementation

OrderService

```
public class OrderService {
   private MySqlOrderRepository repo;

   public OrderService() {
      this.repo = new MySqlOrderRepository();
   }

   public List<Order> getOrders() {
      return repo.findAll();
   }
}
```

Direct Dependency

MySqlOrderRepository

```
public class MySqlOrderRepository {
   public List<Order> findAll() {
      // MySQL specific JDBC code
      Connection conn =
        DriverManager.getConnection(...);
      // ...
   }
}
```

MySQL Database

⚠ Changes in MySQL code, connection logic, or schema impact OrderService too.

Problems with Tight Coupling

Hard to Maintain
A small change in one class requires changes in multiple classes.

Poor Flexibility
Difficult to switch to a different implementation (e.g., PostgreSQL instead of MySQL).

Hard to Test
Dependencies are hard-coded, making unit testing difficult.

Low Reusability
Components cannot be reused in different contexts.

High Change Impact
Changes ripple through the entire system.

Impact on the System

- Changes are expensive and time-consuming
- Strong dependencies limit scalability
- Higher risk of regressions
- Complex to test and debug
- Slower development velocity

Key Takeaway

Tight coupling makes systems brittle and harder to evolve.
Loosely coupled systems are **easier to maintain, test, and extend.**

Goal: Reduce dependencies between components using abstractions and IoC.

PROBLEMS WITH TIGHT COUPLING

As applications grow, tight coupling creates strong dependencies that compound into real engineering costs.

Hard to Maintain

Changes ripple across multiple dependent classes.

Hard to Test

Isolated unit testing needs complex mocks or setups.

Low Flexibility

Difficult to replace or extend functionality.

Poor Reusability

Tightly coupled classes can't be reused elsewhere.

Ripple Effect

A small change can break many unrelated classes.

Slower Development

More time spent tracing dependencies than building.

Difficult to Scale

Dependency management gets complex and error-prone.

Higher Cost

More bugs, more testing, more maintenance overall.

KEY TAKEAWAY Loosely coupled systems are easier to change, test, extend, and scale than tightly coupled ones.

WHAT IS INVERSION OF CONTROL?

IoC transfers control of object creation and wiring to a container — the framework calls your code, not the reverse.

Without IoC

```
public class OrderService {
    private EmailService emailService;

    public OrderService() {
        emailService = new EmailService();
    }
}
// OrderService creates its own dependency
```

With IoC (Spring Container)

```
@Service
public class OrderService {
    private final EmailService emailService;

    @Autowired
    public OrderService(
            EmailService emailService) {
        this.emailService = emailService;
    }
}
```

THE IoC CONTAINER IS RESPONSIBLE FOR:

- Creating objects
- Wiring dependencies
- Managing lifecycle
- Providing configuration

KEY TAKEAWAY IoC lets you focus on business logic while the framework takes care of the plumbing.

Inversion of Control (IoC)

IoC is a design principle where the control of object creation, configuration, and dependency management is transferred from the application code to a container or framework.

spring by VMware Tanzu

What is IoC?

Inversion of Control (IoC)

In traditional programming, your code is in control. With IoC, the framework/container is in control.

You Focus On:

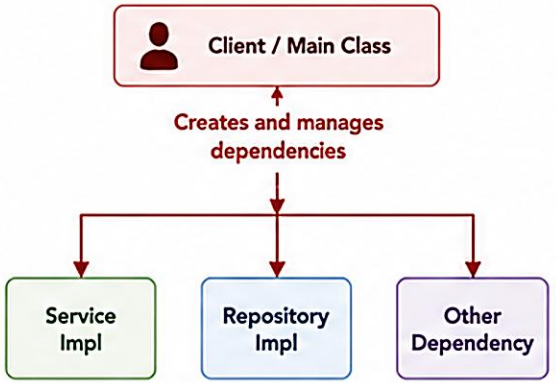
- Business Logic
- Application Features
- High-level Design

Framework handles the rest!

Control Flow Comparison

Without IoC (Traditional)

Application code is in control

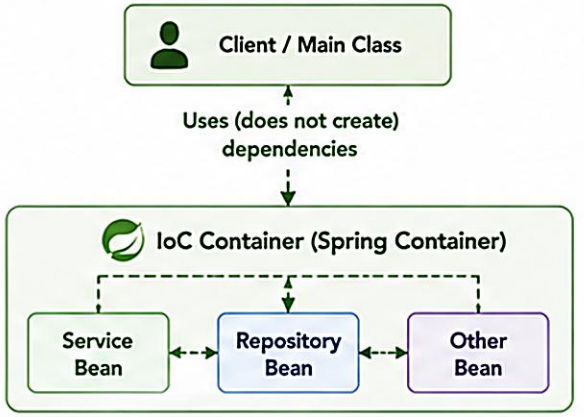


- Objects are created manually using new keyword
- Tight coupling between classes
- Hard to test and maintain

VS

With IoC (Inversion of Control)

Container is in control



- Container creates and manages objects
- Loose coupling
- Easy to test, maintain, and extend

Benefits of IoC

Loose Coupling
Components are independent and easier to change.

Better Testability
Easy to mock dependencies for unit testing.

Reusability
Beans can be reused in different contexts.

Maintainability
Changes are localized, reducing ripple effects.

Increased Productivity
Focus on business logic, not plumbing code.

How IoC Works in Spring

1. **Configuration** — Metadata (XML, Java Config, or Annotations) is provided.
2. **Container Initialization** — Spring IoC Container reads the configuration.
3. **Object Creation** — Container creates the required beans.
4. **Dependency Injection** — Container injects dependencies into beans.
5. **Application Ready** — Beans are ready to use. Control is with the container.

Key Takeaway

IoC lets the framework take control of object lifecycle and dependencies, while you focus on delivering business value.

"Don't call us, we'll call you."
— Hollywood Principle

TRY IT YOURSELF — EXERCISE 1: IoC VERSUS MANUAL WIRING

Reinforces: contrasting IoC with new-coupling for Northstar CRM collaborators — an analysis exercise.

CustomerService.java — the anti-pattern (what NOT to do)

```
public class CustomerService {
    private final CustomerRepository repo = new InMemoryCustomerRepository();
}
```

STEPS (IN notes/ioc-vs-new.md)

Step	What To Do
1 — Spot the Smell	Rewrite the new InMemoryCustomerRepository() anti-pattern above in one sentence; name one problem it causes for swapping persistence later.
2 — Check the Reference	Compare your note to the IoC-vs-new reference table: IoC means the container owns lifecycle; the service just declares needs via constructor parameters.
3 — CRM Fixtures	List three evidence IDs Lab 22 reuses: CUS-1001 (Amina Khan, ACTIVE), CUS-1002 (Ravi Singh, PROSPECT), correlation ID lab-request-001.
4 — Pre-Lab Boundary	Write one line: this exercise prepares for bean wiring only — you will not finish the full Lab 22 starter TODOs here.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-ioc-vs-new.md (workspace: examples/module-22-exercises).

BENEFITS OF IoC (INVERSION OF CONTROL)

IoC, managed by the Spring container, leads to better-designed, easier-to-maintain, enterprise-ready applications.

Loose Coupling

Objects depend on abstractions, not concrete classes.

Better Maintainability

Well-separated responsibilities are easier to modify.

Improved Testability

Dependencies can be mocked or stubbed for unit tests.

Greater Flexibility

Swap implementations without changing client code.

Scalability

Modular design eases growth and new integrations.

Enterprise Ready

Foundation for AOP, transactions, security, and more.

KEY TAKEAWAY IoC promotes loose coupling, reusability, and maintainability — the building blocks of scalable applications.

IoC CONTAINER OVERVIEW (1 OF 2)

Phase 1 — the container reads metadata, then creates and wires the bean objects it describes.

1

Read Metadata

XML or annotations

2

Create Beans

Instantiate bean objects

3

Wire Dependencies

Inject required beans

WHAT HAPPENS IN THIS PHASE

- **Read Metadata —** the container reads bean definitions from XML or annotations.
- **Create Beans —** it instantiates each described bean object.
- **Wire Dependencies —** it injects the required collaborators into each bean.

KEY TAKEAWAY Before a bean can be used, the container must know about it, create it, and wire its dependencies.

IoC Container and Dependency Injection Flow

The IoC Container manages object lifecycle and injects dependencies automatically.

1 Configuration / Component Scan

Spring reads configuration (Java Config / Annotations / XML) and scans components (e.g., @Component, @Service, @Repository)

Input

- @Configuration
- @ComponentScan
- Bean Definitions

2 IoC Container Initialization

IoC Container creates and registers bean definitions (metadata) in the Bean Definition Registry.

Container
Bean Definition Registry

UserService	BeanDefinition
UserRepository	BeanDefinition
DataSource	BeanDefinition
...	

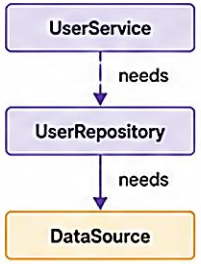
3 Bean Instantiation

The container creates bean instances (objects) for all singleton beans.

Instances Created

userServiceObject
userRepositoryObject
dataSourceObject
...

4 Dependency Resolution

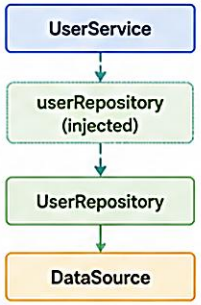


The container analyzes dependencies (via constructor, setter or field) and finds the required beans.

Example

```
@Service
class UserService {
   private final UserRepository
    userRepository;
}
```

5 Dependency Injection



The IoC Container injects the resolved dependencies into the target bean.

Injection Types

- Constructor Injection
- Setter Injection
- Field Injection (not recommended)

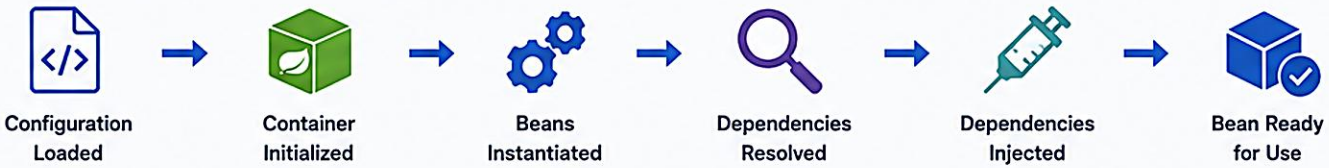
6 Bean Ready for Use

The fully configured bean is ready to use. The container returns the bean when requested.

Outcome

UserService is ready with all dependencies injected and can be used by the application.

How It Works – End to End



Key Benefits

- ✓ Loose Coupling
- ✓ Maintainability
- ✓ Easy Testing
- ✓ Centralized Management
- ✓ Scalability
- ✓ Lifecycle Management

Spring Framework | Inversion of Control (IoC) enables your application to be flexible, testable and maintainable.

Spring IoC Container Overview

The IoC Container is the heart of the Spring Framework.
It creates, configures, manages, and wires the application objects (beans).

spring by VMware Tanzu

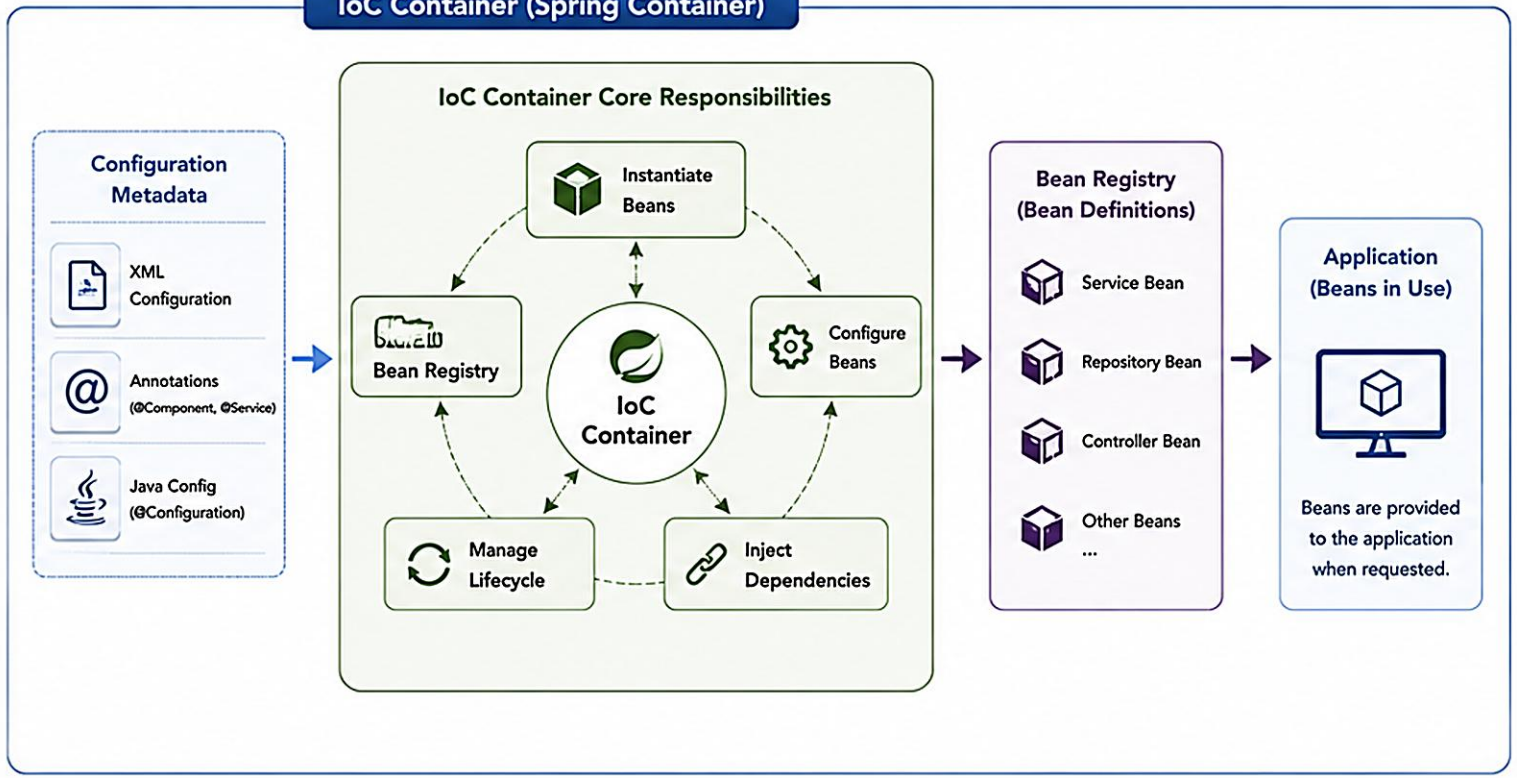
What is IoC Container?

The IoC Container (also known as BeanFactory / ApplicationContext) is responsible for:

- Creating beans
- Configuring beans
- Managing dependencies (Dependency Injection)
- Managing bean lifecycle
- Providing beans to clients when needed

The container uses metadata (annotations, XML, or Java config) to understand and manage your application.

IoC Container (Spring Container)



Key Characteristics

- **Centralized Control** — All objects are created and managed by the container.
- **Loose Coupling** — Objects don't create or look up dependencies.
- **Dependency Injection** — Dependencies are provided by the container.
- **Lifecycle Management** — Container manages the full lifecycle of beans.
- **Configurable** — Supports XML, Annotations, and Java Configuration.
- **Scalable & Extensible** — Easily extendable for enterprise applications.

Result
Better design, easier testing, and highly maintainable applications.

How the IoC Container Works (High-Level Flow)

1. **Load Configuration** — Container reads configuration (metadata) from XML, annotations, or Java config.
2. **Scan & Register** — Container scans and registers bean definitions in the bean registry.
3. **Instantiate Beans** — Container creates objects (beans) as per the bean definitions.
4. **Inject Dependencies** — Container injects the required dependencies into the beans.
5. **Ready to Use** — Beans are fully initialized and made available to the application.
6. **Manage Lifecycle** — Container manages lifecycle, scopes, and destruction of beans.

BeanFactory vs ApplicationContext

BeanFactory	ApplicationContext
• Lightweight container • Lazy loading • Basic IoC support	• Advanced container • Eager initialization • Enterprise features (AOP, i18n, events, etc.)

VS

IoC CONTAINER OVERVIEW (2 OF 2)

Phase 2 — the container configures each bean, makes it available, and later manages its lifecycle.

1

Configure Beans

Apply properties & settings

2

Ready to Use

Fully initialized beans

3

Manage Lifecycle

Cleanup and destruction

Container Type	Characteristics
BeanFactory	Basic container; lazy initialization; used in resource-constrained environments.
ApplicationContext	Enterprise-ready superset; eager initialization; supports AOP, events, i18n, validation.

KEY TAKEAWAY The IoC Container is Spring's backbone — it promotes loose coupling, modularity, and testability.

WHAT IS DEPENDENCY INJECTION?

DI is a design pattern where an object receives its dependencies from the container instead of creating them.

Constructor Injection

Provided via the constructor. Recommended — ensures immutability and testability.

Setter Injection

Provided via setter methods. Good for optional, changeable dependencies.

Field Injection

Injected directly into fields via @Autowired. Easy but hard to test — avoid in production.

BENEFITS OF DEPENDENCY INJECTION

Reduces Coupling | **Improves Testability** | **Increases Flexibility** | **Enhances Maintainability**

HOW IT WORKS

- The IoC Container creates and configures the dependency.
- The container injects the dependency into the target object.
- The dependent object is ready to use.

KEY TAKEAWAY Dependency Injection lets objects stay loosely coupled while the IoC container handles the wiring.

Dependency Injection (DI) in Spring

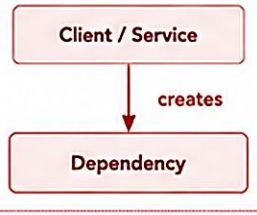
Dependency Injection is a design pattern where the IoC container provides the dependencies that a bean needs, instead of the bean creating them itself.

spring by VMware Tanzu

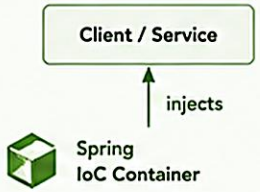
What is Dependency Injection?

DI is a key part of IoC.
It allows components (beans) to receive their dependencies from the container rather than creating them.

Without DI (Manual Dependency)

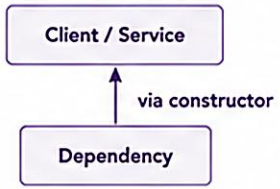


With DI (Container Injects)



Types of Dependency Injection

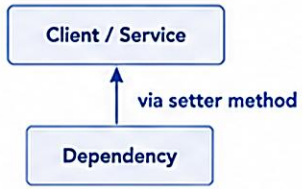
1. Constructor Injection (Recommended)



- Dependencies are provided through the constructor.
- Ensures immutability and required dependencies.
- Preferred for mandatory dependencies.

```
@Service
public class OrderService {
   private final PaymentService paymentService;

   public OrderService(PaymentService paymentService) {
      this.paymentService = paymentService;
   }
}
```

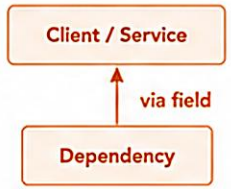
2. Setter Injection



- Dependencies are injected using setter methods.
- Useful for optional dependencies.
- Allows changes after object creation.

```
@Service
public class OrderService {
   private PaymentService paymentService;

   @Autowired
   public void setPaymentService(PaymentService
    paymentService) {
      this.paymentService = paymentService;
   }
}
```

3. Field Injection (Not Recommended)



- Dependencies are injected directly into fields.
- Makes testing difficult.
- Hides dependencies and tightens coupling.

```
@Service
public class OrderService {
   @Autowired
   private PaymentService paymentService;
}
```

Injection Best Practices

Prefer Constructor Injection
Promotes immutability and makes dependencies explicit.

Keep Dependencies Minimal
Inject only what is needed.

Use Interfaces
Depend on abstractions, not implementations.

Avoid Field Injection
Makes code harder to test and maintain.

Design for Testability
Constructor injection makes unit testing easier.

How DI Works in Spring

1. **Configuration** — Metadata (XML, Annotations, Java Config) is loaded.
2. **Bean Creation** — IoC Container creates bean instances.
3. **Dependency Resolution** — Container finds required dependencies.
4. **Injection** — Dependencies are injected into the bean.
5. **Bean Ready** — Bean is fully initialized and ready to use.

Benefits of DI

Loose Coupling | High Testability | Better Reusability | Easy Maintenance | Scalability

Key Takeaway: Dependency Injection, powered by the Spring IoC container, helps build loosely coupled, testable, and maintainable enterprise applications.

Real-World Use Case

In enterprise applications (e.g., banking, e-commerce), DI allows easy swapping of implementations (e.g., different payment providers) without changing business logic.

CONSTRUCTOR INJECTION

The preferred way to inject dependencies in Spring — required dependencies provided through the constructor.

WHY CONSTRUCTOR INJECTION?

- **Mandatory Dependencies —** ensures required dependencies are provided at creation time.
- **Immutability —** the dependency field can be declared final.
- **Easy to Test —** dependencies are explicit; no reflection needed for unit tests.
- **No Partial State —** the object is fully initialized when constructed.
- **Best Practice —** recommended by the Spring team as the default choice.

HOW IT WORKS

1

Creates Dependency

EmailService bean created

2

Injects via Constructor

Passed into OrderService()

3

Bean Ready to Use

OrderService fully wired

KEY TAKEAWAY Constructor injection is Spring's recommended default — it produces robust, testable, maintainable code.

Constructor Injection in Spring

Dependencies are provided to a bean through its constructor.
It is the recommended way of Dependency Injection.

spring by VMware Tanzu

What is Constructor Injection?

The IoC container provides the required dependencies to a bean by calling its constructor with those dependencies.

How it works

All required dependencies are provided at the time of object creation.

Why It Is Recommended

- **Ensures immutability** — Dependencies are set once and cannot be changed.
- **Prevents null values** — Required dependencies are always provided.
- **Easier to test** — Makes unit testing simple by explicitly providing dependencies.
- **Clear dependencies** — All dependencies are visible in the constructor.

Example: OrderService depends on PaymentService and CustomerRepository

1. Dependency Beans

```
@Service
public class PaymentService {
  public void processPayment() {
    System.out.println("Payment processed");
  }
}

@Repository
public class CustomerRepository {
  public Customer findById(Long id) {
    return new Customer(id, "John Doe");
  }
}
```

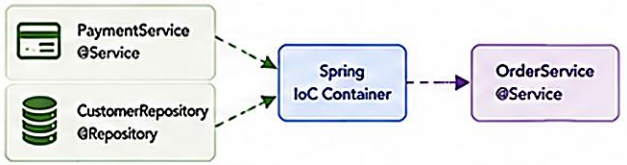
2. Bean with Constructor Injection

```
@Service
public class OrderService {
   private final PaymentService paymentService;
   private final CustomerRepository customerRepository;

   // Constructor Injection
   public OrderService(PaymentService paymentService,
            CustomerRepository customerRepository) {
      this.paymentService = paymentService;
      this.customerRepository = customerRepository;
   }

   public void placeOrder(Long customerId, double amount) {
     Customer customer = customerRepository.findById(customerId);
     paymentService.processPayment();
     System.out.println("Order placed for " + customer.getName()
                        + " amount: " + amount);
   }
}
```

3. Configuration (Component Scanning)

```
@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {
  // Spring will automatically detect
  // and create the beans
}
```

Constructor Injection in Action



Dependencies are injected through the constructor at runtime.

Key Takeaway

Constructor Injection leads to immutable, testable, and maintainable code.
It is the preferred way to achieve Dependency Injection in Spring.

Analogy

Constructor Injection is like building a car: all required parts (dependencies) are provided upfront to make the car ready to run.

CONSTRUCTOR INJECTION — EXAMPLE

A complete constructor-injected service, plus the equivalent Java configuration.

OrderService.java

```
@Service
public class OrderService {
    private final EmailService emailService;

    // Constructor Injection
    public OrderService(EmailService emailService) {
        this.emailService = emailService;
    }

    public void placeOrder() {
        emailService.sendEmail("Order placed");
    }
}
```

AppConfig.java (Java Config)

```
@Configuration
public class AppConfig {
    @Bean
    public OrderService orderService(
            EmailService emailService) {
        return new OrderService(emailService);
    }
}
```

KEY POINTS

- Dependencies pass through the constructor and are final.
- The class cannot be built without its dependencies.
- Since Spring 4.3+, a single constructor needs no @Autowired.

KEY TAKEAWAY If a class has only one constructor, Spring wires it automatically — no @Autowired required.

TRY IT YOURSELF — EXERCISE 2: CONSTRUCTOR INJECTION

Reinforces: documenting why constructor injection with final fields is the Northstar standard.

STYLE COMPARISON

Style	Verdict
Constructor + final	Preferred — dependencies are required and explicit; the field stays immutable after construction.
Setter injection	Optional only — reserve for a dependency that may be absent or changed later.
Field @Autowired	Avoid as the primary pattern — hides dependencies and blocks pure unit testing.

FILL-IN-THE-BLANK ANSWER KEY (notes/constructor-di.md)

```
Northstar prefers CONSTRUCTOR injection because dependencies
are REQUIRED (explicit) and fields can be FINAL (immutable).

// Constructor signature only (no method bodies):
CustomerService(CustomerRepository repo, NotificationService notifier)
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-constructor-injection.md (a pure unit test can `new CustomerService(fakeRepo, fakeNotifier)` without starting Spring).

SETTER INJECTION

Dependencies are provided through setter methods after construction — useful for optional dependencies.

WHY SETTER INJECTION?

- **Optional Dependencies —** useful when a dependency is not mandatory.
- **Flexibility —** dependencies can be changed or updated after creation.
- **Reusability —** the same object can be reused with different configurations.
- **Easier Large Configuration —** helpful in XML configs with many properties.
- **Legacy Integration —** works with code that expects a no-arg constructor.

HOW IT WORKS

1. **Container Creates Bean**
   No-arg constructor used
2. **Calls Setter Method**
   setEmailService() invoked
3. **Bean Ready to Use**
   Dependency now attached

KEY TAKEAWAY Setter injection offers flexibility for optional dependencies, but constructor injection stays preferred.

Setter Injection in Spring

Dependencies are provided to a bean through setter methods after the bean is created by the container.
It allows optional dependencies and can be changed after object creation.

spring by VMware Tanzu

What is Setter Injection?

The IoC container first creates the bean using a no-argument constructor, then injects the dependencies by calling setter methods.

How it works

Dependencies can be injected and replaced after the bean is created.

When to Use

- When dependencies are optional or may not always be required.
- When you want to change or reconfigure dependencies at runtime.
- When working with frameworks that require a no-arg constructor.
- Useful in testing and flexible configurations.

Example: OrderService depends on PaymentService and CustomerRepository

1. Create Bean
Container creates the OrderService bean using no-arg constructor.
OrderService

2. Set Dependencies
Container calls setter methods to inject dependencies.
- setPaymentService(...)
- setCustomerRepository(...)

3. Bean Ready
All dependencies are set. Bean is ready to use.
OrderService

4. Use Bean
Bean is retrieved from the container and used by the application.

1. Bean Class (Dependent Bean)

```
@Service
public class OrderService {
     private PaymentService paymentService;
     private CustomerRepository customerRepository;

  // No-arg constructor (required for setter injection)
  public OrderService() {
       System.out.println("OrderService created");
    }
  // Setter for PaymentService
  @Autowired
    public void setPaymentService(PaymentService paymentService) {
       this.paymentService = paymentService;
    }
  // Setter for CustomerRepository
  @Autowired
    public void setCustomerRepository(CustomerRepository repo) {
       this.customerRepository = repo;

  public void placeOrder(Long customerId, double amount) {
     // use paymentService and customerRepository
}
```

2. Dependency Beans

```
@Service
public class PaymentService {
  public void processPayment() {
    System.out.println("Payment processed");
  }
}
```

```
@Repository
public class CustomerRepository {
  public Customer findById(Long id) {
    return new Customer(id, "John Doe");
  }
}
```

3. Configuration (Component Scanning)

```
@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {
    // Spring will:
    // 1. Create beans
    // 2. Call setter methods to inject
         dependencies automatically
}
```

Setter Injection in Action

Container creates OrderService and injects dependencies via setter methods.

Advantages

- **Supports Optional Dependencies** — Not all dependencies need to be set.
- **Reconfigurable** — Dependencies can be changed or re-injected.
- **Easy Maintenance** — Setter methods improve readability and flexibility.
- **Better for Testing** — Mocks/alternatives can be injected easily.
- **Supports Framework Requirements** — Some frameworks require a no-arg constructor.

Key Takeaway

Setter Injection provides flexibility by allowing dependencies to be set after object creation.
It is ideal for optional or changeable dependencies and improves testability.

Constructor vs Setter Injection

Constructor Injection	Setter Injection
Dependencies set via constructor at object creation	Dependencies set via setter methods after object creation
Ensures immutability	Allows optional dependencies
Mandatory dependencies	Dependencies can be changed
Preferred for required dependencies	Useful for reconfiguration

VS

Use Case Example

In a banking application, the *PaymentService* may vary based on configuration. Setter Injection allows you to swap the payment provider without changing code.

SETTER INJECTION — EXAMPLE

Dependency provided through a setter, plus the equivalent XML configuration.

OrderService.java

```
public class OrderService {
    private EmailService emailService;

    // Setter Injection
    @Autowired
    public void setEmailService(
            EmailService emailService) {
        this.emailService = emailService;
    }
}
```

XML Configuration

```
<bean id="orderService"
      class="com.example.OrderService">
    <property name="emailService"
              ref="emailService" />
</bean>
```

BEST USE CASES

- Optional deps
- Configurable props
- Runtime-changeable
- Legacy code

KEY TAKEAWAY Use setter injection for optional or reconfigurable dependencies — not as the default choice.

FIELD INJECTION

Dependencies are injected directly into fields via reflection — the least recommended DI style in Spring.

ADVANTAGES

- Less boilerplate — no constructor or setter methods.
- Quick to implement for small demos or prototypes.
- Simple, compact class definitions.

DISADVANTAGES

- Not test-friendly — objects can't be built outside the container.
- Hidden dependencies — not visible in the class definition.
- Breaks encapsulation by injecting into private fields directly.

The Spring team recommends constructor injection over field injection for production code.

KEY TAKEAWAY Field injection works, but it hides dependencies and makes testing harder — prefer constructor injection.

Field Injection in Spring

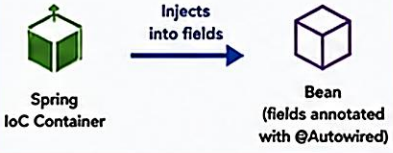
Dependencies are injected directly into fields using @Autowired.
It is convenient but not the recommended way.

spring by VMware Tanzu

What is Field Injection?

The IoC container injects dependencies directly into fields of a bean using @Autowired.

How it works



Container uses reflection to set the value of dependencies into fields after the bean is created.

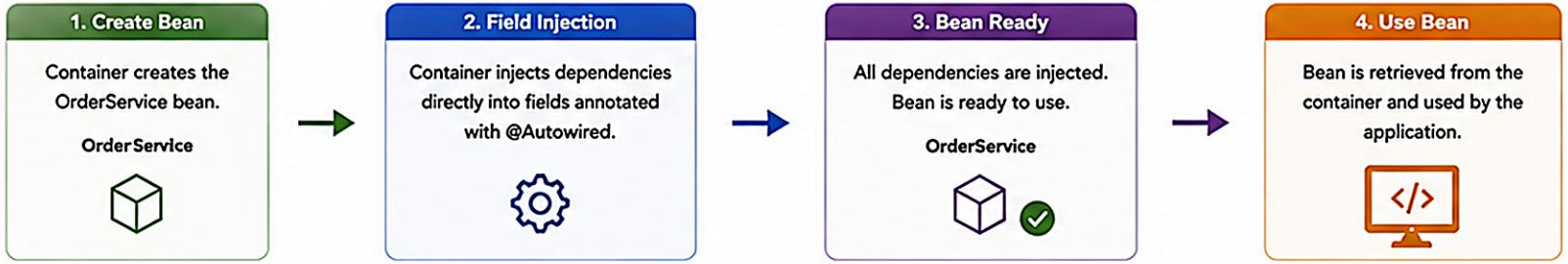
When to Use

- Quick and simple for prototypes or small applications.
- When you control the class and want less boilerplate code.
- Useful in testing with mocks (easy to inject).

When to Avoid

- Harder to unit test (tight coupling).
- Cannot make dependencies final.
- Hidden dependencies reduce clarity.
- Not recommended for production code in large systems.

Example: OrderService depends on PaymentService and CustomerRepository



1. Bean Class (Dependent Bean)

```
@Service
public class OrderService {
   @Autowired
    private PaymentService paymentService;

   @Autowired
   private CustomerRepository customerRepository;

   public void placeOrder(Long customerId, double amount) {
      Customer customer = customerRepository.findById(customerId);
      paymentService.processPayment();
      System.out.println("Order placed for " + customer.getName()
                + " amount: " + amount);
   }
}
```

2. Dependency Beans

```
@Service
public class PaymentService {
   public void processPayment() {
      System.out.println("Payment processed");
   }
}

@Repository
public class CustomerRepository {
   public Customer findById(Long id) {
      return new Customer(id, "John Doe");
   }
}
```

3. Configuration (Component Scanning)

```
@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {
   // Spring will:
   // 1. Scan components
   // 2. Create beans
   // 3. Inject dependencies into fields
}
```

Field Injection in Action

Container uses reflection to set values into fields annotated with @Autowired.

Field Injection vs Other Injection

Aspect	Constructor Injection	Setter Injection	Field Injection
How dependencies are set	Via constructor	Via setter methods	Directly into fields
Immutability	✅ Yes	❌ No	❌ No
Testability	✅ High	✅ Medium	❌ Low
Visibility of dependencies	✅ Explicit	✅ Explicit	❌ Hidden
Recommended	✅ Best Practice	✅ Acceptable	❌ Not Recommended

Key Takeaway

Field Injection works, but hides dependencies and makes testing difficult. Prefer Constructor Injection for better design, testability, and maintainability.

"Use Field Injection only when simplicity is more important than design."

Real-World Use Case

In a small prototype, you can quickly inject repositories or services into fields to save time.

In enterprise applications, prefer constructor injection to build robust and maintainable systems.

FIELD INJECTION — EXAMPLE

Dependency injected directly into a private field via @Autowired.

OrderService.java

```
public class OrderService {

    @Autowired
    private EmailService emailService;

    public void placeOrder() {
        emailService.sendEmail("Order placed");
    }
}
```

KEY POINTS

- @Autowired is applied directly to the field.
- The container injects the dependency via reflection.
- Cannot be unit tested without a Spring context.
- Should be avoided in production code.

BEST USE CASES (LIMITED)

Quick prototypes | Legacy applications | Minimal demo code

KEY TAKEAWAY Use constructor injection for mandatory deps and setter injection for optional ones — field injection is a last resort.

INJECTION BEST PRACTICES

Practical guidelines for using Spring dependency injection effectively in enterprise code.

#	Practice	Why It Matters
1	Prefer Constructor Injection	Default choice — required, immutable, testable dependencies.
2	Use Setter Injection for Optional Deps	Only when a dependency may be absent or change later.
3	Avoid Field Injection	Hides dependencies and blocks testing outside Spring.
4	Program to Interfaces	Depend on abstractions; let Spring supply the implementation.
5	Keep Beans Focused	Single responsibility — avoid god classes.
6	Use @Qualifier When Needed	Disambiguate when multiple beans share a type.
7	Avoid Circular Dependencies	Redesign rather than field-inject around a cycle.
8	Test With Mocked Dependencies	Verify business logic in isolation from Spring.

KEY TAKEAWAY Use constructor injection by default, keep dependencies explicit, and test business logic in isolation.

WHAT IS A SPRING BEAN?

Any object created, configured, wired, and managed by the Spring IoC container.

Container-Managed

The IoC container creates and manages the bean.

Configured as Metadata

Defined via XML, Java config, or annotations.

Dependencies Injected

The container injects required collaborators.

Lifecycle Managed

Container controls init, use, and destroy phases.

Reusable & Scalable

Promotes reuse, loose coupling, easy testing.

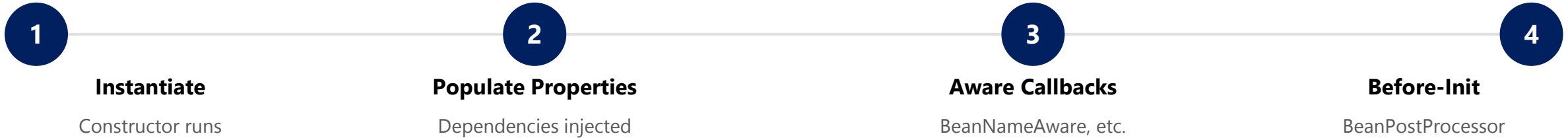
TYPES OF SPRING BEANS (BY SCOPE)

Scope	Description
Singleton (default)	One shared instance per Spring container.
Prototype	A new instance every time the bean is requested.
Request / Session	Web-aware scopes — one instance per HTTP request or session.

KEY TAKEAWAY A Spring Bean is an object the container creates, wires, and manages — promoting loose coupling and reuse.

BEAN LIFECYCLE OVERVIEW (1 OF 2)

Phase 1 — the container brings a bean into existence and prepares it for initialization.



WHAT HAPPENS IN THIS PHASE

- **Instantiate —** the container calls the bean's constructor to create the raw object.
- **Populate Properties —** constructor or setter injection wires the bean's dependencies.
- **Aware Callbacks —** BeanNameAware, BeanFactoryAware, and similar callbacks run if implemented.
- **Before-Init —** any registered BeanPostProcessor.postProcessBeforeInitialization() runs.

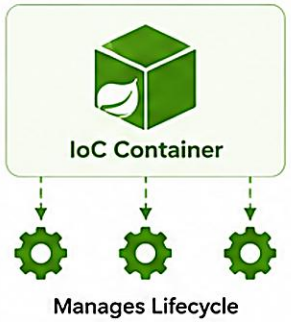
KEY TAKEAWAY Before a bean is ever used, the container must create it, wire it, and run any pre-init callbacks.

Spring Bean Lifecycle

From creation to destruction – managed by the IoC Container

OVERVIEW

The Spring IoC container manages the full lifecycle of a bean – from instantiation, initialization, usage, to destruction.

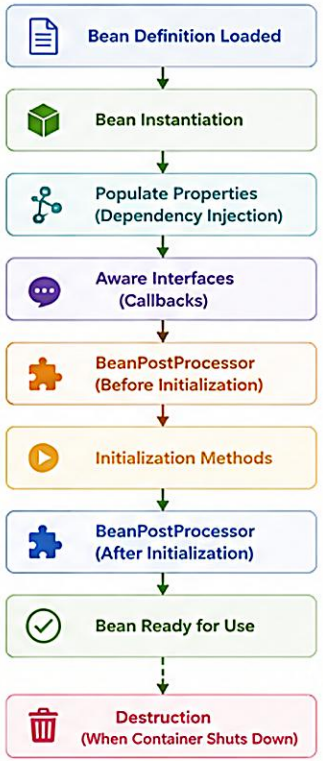


KEY CALLBACK INTERFACES (OPTIONAL HOOKS)

BeanNameAware	Aware of bean name
InitializingBean	`afterPropertiesSet()`
DisposableBean	`destroy()`
@PostConstruct	Custom init method
@PreDestroy	Custom destroy method

STEP	WHAT HAPPENS
1 Bean Definition Loaded	Container reads configuration metadata (XML, Java Config, Annotations) and registers bean definitions.
2 Bean Instantiation	The container creates an instance of the bean using the default constructor (or factory method).
3 Populate Properties (Dependency Injection)	The container sets properties and resolves dependencies (constructor, setter or field injection).
4 Aware Interfaces (Callbacks)	If the bean implements Aware interfaces (BeanNameAware, BeanFactoryAware, etc.), the corresponding methods are called.
5 BeanPostProcessor (Before Initialization)	All registered BeanPostProcessors are invoked before initialization (postProcessBeforeInitialization).
6 Initialization Methods	• If InitializingBean is implemented – `afterPropertiesSet()` is called • If custom `init-method` is defined – it is invoked • If `@PostConstruct` is used – method is called
7 BeanPostProcessor (After Initialization)	All registered BeanPostProcessors are invoked after initialization (postProcessAfterInitialization).
8 Bean Ready for Use	The fully initialized bean is returned by the container and ready to be used by the application.
9 Destruction (When Container Shuts Down)	• If `@PreDestroy` is used – method is called • If `DisposableBean` is implemented – `destroy()` is called • If custom `destroy-method` is defined – it is invoked

LIFECYCLE FLOW DIAGRAM



NOTES

- ✓ Singleton beans are created once and cached.
- ✓ Protopable beans are created on every request.
- ✓ Destruction callbacks are invoked only for singleton beans.

Spring manages the complete lifecycle – You focus on business logic!

spring by VMware

BEAN LIFECYCLE OVERVIEW (2 OF 2)

Phase 2 — the bean is initialized, used by the application, and eventually destroyed.

1 **Initialize**

afterPropertiesSet() / init

2 **Ready to Use**

Bean available in context

3 **Destroy**

@PreDestroy / destroy

Lifecycle Callbacks Example

```
@Component
public class OrderService {
    @PostConstruct
    void init() {
        // setup logic — runs once after dependencies are injected
    }
    @PreDestroy
    void shutdown() {
        // cleanup logic — runs before the bean is destroyed
    }
}
```

KEY TAKEAWAY Use @PostConstruct for setup and @PreDestroy for cleanup — proper lifecycle management prevents resource leaks.

TRY IT YOURSELF — EXERCISE 3: BEAN LIFECYCLE CALLBACKS

Reinforces: predicting when @PostConstruct and @PreDestroy fire for a singleton CustomerService.

STEPS

Step	What To Do
1 — Order the Phases	Number these five phases: inject dependencies → create bean → @PostConstruct → serve requests → @PreDestroy.
2 — Check the Reference	Correct order: create → inject dependencies → @PostConstruct → serve requests → @PreDestroy.
3 — Evidence Plan	Write the log lines expected once per context start/stop — no secrets or PII in logs (Module 20 rules apply).
4 — Scope Note	State that a bean's default scope is singleton unless a different scope is explicitly annotated.

CustomerService.java — the callbacks you predicted

```
@Service
public class CustomerService {
    @PostConstruct
    void init() { /* log: CustomerService ready (no PII) */ }
    @PreDestroy
    void shutdown() { /* log: CustomerService shutting down */ }
}
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-lifecycle-notes.md.

BEAN SCOPES IN SPRING

A bean's scope determines how many instances the container creates and how long each one lives.

Scope	Instances	Typical Use Case
singleton (default)	1 per container	Stateless services, repositories, configuration beans.
prototype	New instance per request	Stateful, non-shared, or task-specific objects.
request (web-aware)	1 per HTTP request	Web controllers, request-scoped data.
session (web-aware)	1 per HTTP session	User session data, shopping cart.

DECLARING SCOPE

`@Scope("singleton")` `@Scope("prototype")` `@Scope(value="request", proxyMode=...)`

request, session, application, and websocket scopes are web-aware and only work in a web-aware Spring application.

KEY TAKEAWAY Choose singleton for shared, stateless beans and prototype for stateful or non-shared beans.

Spring Bean Scopes

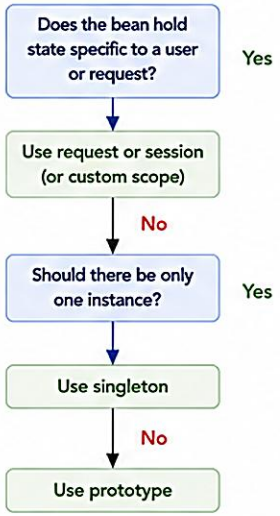
A scope defines the lifecycle and visibility of a bean within the Spring IoC container.

spring by VMware Tanzu

What is Scope?

Scope determines how many instances of a bean exist in the container, how long they live, and where they are visible.

Scope Decision Guide



Built-in Scopes

Scope	Description	Lifecycle	Instance Per	Use Case	Diagram
singleton (Default)	Only one instance of the bean is created per Spring IoC container.	Created at container startup (or first request) and lives until container shutdown.	Spring IoC Container	• Stateless services • DAOs • Components that can be shared	Container
prototype	A new instance of the bean is created every time it is requested.	Created when requested and destroyed when no longer referenced.	Every Bean Request	• Stateful objects • Objects with dependency on external resources	Container
request	One instance per HTTP request.	Created at the beginning of request and destroyed at the end of request.	Each HTTP Request (Per Thread)	• Controllers • Request-scoped data • Form backing objects	HTTP Request
session	One instance per HTTP session.	Created at the beginning of session and destroyed when the session expires or invalidates.	Each HTTP Session	• User preferences • Shopping cart • User-specific data	HTTP Session
application	One instance per ServletContext (lifecycle of the web application).	Created at application startup and lives until application shutdown.	Web Application (ServletContext)	• Shared resources • Application-level configuration	Web Application
websocket	One instance per WebSocket session.	Created at the start of WebSocket session and destroyed when the session ends.	Each WebSocket Session	• WebSocket handlers • Real-time chat • Live updates	WebSocket Session

Key Points

- singleton is the default scope in Spring.
- Scopes other than singleton are available only in a web-aware Spring application.
- request, session and websocket are bound to the current thread / context.
- prototype beans are not managed after creation (destroy callbacks are not called automatically).
- You can define custom scopes to suit your application needs.

Example

```
@Component
@Scope("prototype")
public class PaymentService {
   // new instance on every request
}

@Bean
@Scope("request")
public MyRequestData requestData() {
   return new MyRequestData();
}
```

Custom Scopes

Implement org.springframeork.beans.factory.config.

Implement Methods

- Object get(String name, ObjectFactory<?> objectFactory)
- Object remove(String name)
- void registerDestructionCallback(String name, Runnable callback)
- Object resolveContextualObject(String key)
- String getConversationId()

Register the scope

```
ConfigurableBeanFactory
beanFactory = ...;
beanFactory.registerScope
("custom", new CustomScope());
```

Scope Annotations

SINGLETON SCOPE

The default Spring scope — only one instance is created per container and shared across the application.

Single Instance

Only one instance exists for the entire container.

Shared Everywhere

The same instance is injected into every dependent bean.

Created at Startup

Instantiated eagerly when the container starts, by default.

Best for Stateless Beans

Ideal for services, repositories, and utilities.

Example — Singleton by Default

```
@Service
public class OrderService {
    public void placeOrder() { /* ... */ }
}
// No @Scope needed — singleton is the default
```

- Stored in the container's singleton cache.
- All clients receive the same instance.
- Thread-safe only if internal state is handled carefully.

KEY TAKEAWAY Use singleton scope for stateless, thread-safe beans shared across the application — Spring's default scope.

Singleton Scope in Spring

Singleton is the default scope in Spring.
Only one instance of the bean is created per Spring IoC container.

spring by VMware Tanzu

What is Singleton?

The Spring IoC container creates only one instance of a bean for the entire application context. All requests for that bean return the same instance.

1 Instance shared across the application

Key Characteristics

Single Instance
Only one instance per Spring IoC container.

Shared
Shared by all requesting components.

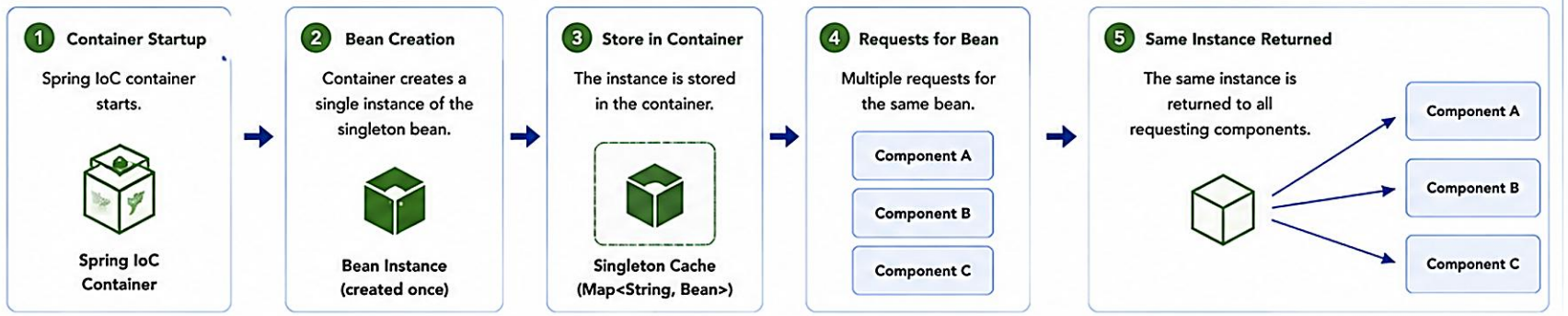
Lifecycle
Created at container startup (by default, lazy-init can delay creation).

Thread-Safe
Ensure your singleton beans are thread-safe.

Example Use Cases

- Stateless services
- Caches
- Configuration properties
- Utility classes
- Service layer components

How Singleton Scope Works

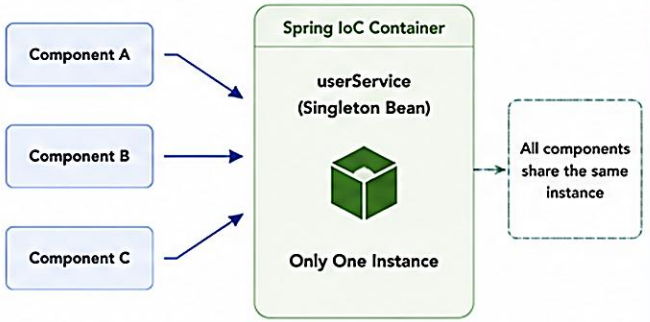


Example

```
@Service
public class UserService {
    public void doWork() {
        System.out.println("Doing work");
    }
}
```

```
<!-- Singleton is default, no scope needed -->
<bean id="userService" class="com.example.UserService" />

<!-- Explicitly defining singleton -->
<bean id="userService" class="com.example.UserService"
      scope="singleton" />
```

Runtime Behavior



Important Notes

Default Scope
If no scope is specified, the bean is singleton.

Per Container
One instance per Spring ApplicationContext. Different contexts have different instances.

Stateful Beans
Avoid storing user/session specific data in singleton beans.

Lazy Initialization
Use `lazy-init="true"` to delay creation until first use.

Singleton vs Other Scopes

Scope	Instances	Created	Shared	Use Case
singleton (default)	1 per container	At startup (eager by default)	Shared by all	Stateless services, shared resources
prototype	New instance per request	When requested	Not shared	Complex objects, non-shared state
request	1 per HTTP request	At start of request	Within same request	Request-scoped data
session	1 per HTTP session	At start of session	Within same session	User session data
application	1 per ServletContext	At application startup	Shared in web app	Application-wide data
websocket	1 per WebSocket session	At start of WebSocket session	Within same WS session	WebSocket handlers

Summary

Singleton is ideal for stateless, thread-safe beans that are expensive to create or need to be shared across the application. It is the most commonly used scope in Spring applications.

PROTOTYPE SCOPE

A new instance is created every time the bean is requested — the container does not cache it.

New Instance Every Time

A fresh instance is created on every request.

Not Cached

The container does not store or reuse the instance.

Independent State

Each instance has its own, isolated state.

No Auto-Destroy

Container skips destroy callbacks; client manages cleanup.

Example — @Scope("prototype")

```
@Component
@Scope("prototype")
public class ReportGenerator {
    private final UUID id = UUID.randomUUID();
}
// Each request gets a different instance/id
```

- Use for stateful beans or per-request/user data.
- Retrieve via context.getBean(...) each time.
- p1 != p2 for two separate lookups.

KEY TAKEAWAY Use prototype scope when each client needs its own fresh, stateful instance.

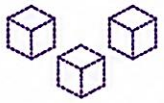
Prototype Scope in Spring

A new instance of the bean is created every time it is requested from the Spring IoC container.

spring by VMware Tanzu

What is Prototype?

In prototype scope, the Spring container creates a new instance of the bean every time it is requested. The instance is not cached by the container.



New Instance created for each request

Key Characteristics

New Instance Each Time
A new instance is created every time the bean is requested.

Not Shared
Instances are not shared between components.

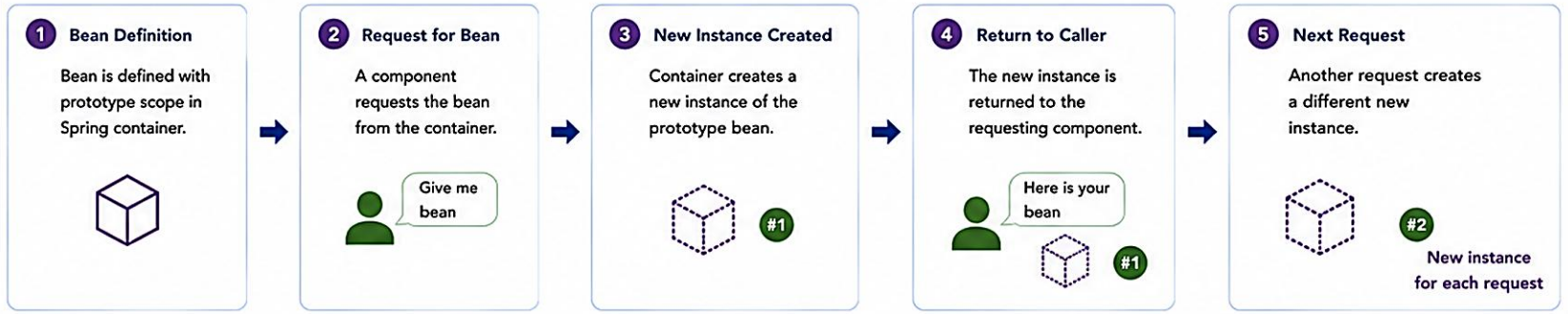
Lifecycle
Container creates, but it does not manage the full lifecycle after creation.

Use destroy() manually
Spring container does not call destroy methods automatically.

When to Use

- For stateful objects
- When each request needs separate data
- For non-thread-safe objects
- When cost of creation is acceptable

How Prototype Scope Works

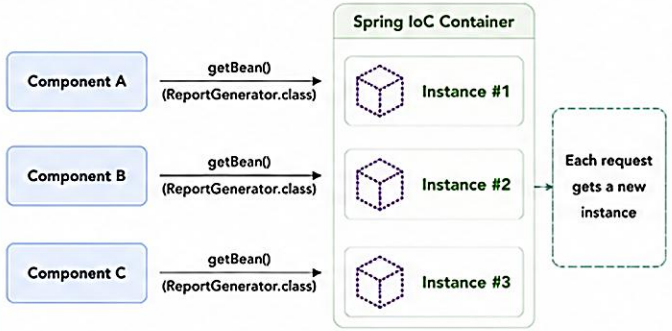


Example

```
@Scope("prototype")
@Component
public class ReportGenerator {
   private String id = UUID.randomUUID().toString();

   public String getId() {
      return id;
   }
}
```

Every time `ReportGenerator` is requested, a new instance with a `different id` is returned.

Runtime Behavior



Important Notes

Not the Default
Prototype is not the default scope. Default scope is singleton.

No Lifecycle Management
Spring container does not manage the full lifecycle after creation.

Destruction
You must call destroy() manually if required.

Dependency Injection
When a singleton depends on a prototype bean, the same instance may be injected unless special handling is used (e.g., Provider, ObjectFactory, lookup method).

Prototype vs Other Scopes

Scope	Instances	Created	Shared	Lifecycle	Typical Use Case
prototype	New instance each time	When requested	No	Not managed after creation	Stateful objects, request-specific data
singleton (default)	One instance per container	At container startup	Yes	Managed by container	Stateless services, caches, configuration
request	One instance per request	At start of HTTP request	Within same request	Managed by container	Request-scoped data, controllers
session	One instance per session	At start of HTTP session	Within same session	Managed by container	User session data
application	One instance per application (ServletContext)	At application startup	Yes	Managed by container	Application-wide data
websocket	One instance per WebSocket session	At start of WebSocket session	Within same WS session	Managed by container	WebSocket handlers, live chat

Summary

Prototype scope creates a new instance of the bean every time it is requested. It is ideal for stateful or request-specific objects where each consumer needs its own instance.

COMPONENT SCANNING OVERVIEW

Spring automatically detects and registers annotated classes as beans, eliminating manual bean declarations.

1. **Base Package Specified** — Point Spring at a root package
2. **Classpath Scan** — Sub-packages scanned too
3. **Component Detection** — Stereotype classes found
4. **Bean Registration** — Registered in the context
5. **Ready for Injection** — Beans available for DI

Enabling Component Scanning

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            DemoApplication.class, args);
    }
}
```

STEREOTYPES DETECTED

- **@Component —** generic Spring-managed component.
- **@Service —** business/service layer.
- **@Repository —** data access layer (DAO).
- **@Controller —** web/MVC layer.

KEY TAKEAWAY Component scanning reduces manual configuration and enables a clean, modular application architecture.

Spring Component Scanning

Component scanning is the process of automatically detecting Spring components in the classpath and registering them as beans in the ApplicationContext.

spring by VMware Tanzu

What is Component Scanning?

Spring scans the specified base package and its sub-packages for classes annotated with stereotype annotations and registers them as beans in the IoC container.

- Automatic bean detection
- Reduces XML configuration
- Improves productivity

Stereotype Annotations Detected

- **@Component** — Generic stereotype for any Spring bean.
- **@Service** — Used for service layer components.
- **@Repository** — Used for DAO / repository layer.
- **@Controller** — Used for web layer components.
- **@RestController** — Combination of @Controller and @ResponseBody.

Benefits

- Automatic detection and registration of beans
- Supports modular and layered architecture
- Easier maintenance and scalability
- Works seamlessly with Spring Boot auto-configuration

How Component Scanning Works

1. **Specify Base Package** — You define the base package to scan using annotations or XML.
2. **Scan Classpath** — Spring scans the classpath for classes in the base package and its sub-packages.
3. **Detect Components** — Classes annotated with stereotype annotations are identified as components.
4. **Register as Beans** — Detected components are registered as beans in the ApplicationContext.
5. **Use Beans** — Beans are available for dependency injection and use in the application.

Example Project Structure

```
src/main/java
└── com.example
    ├── controller
    │   └── UserController.java
    ├── service
    │   └── UserService.java
    ├── repository
    │   └── UserRepository.java
    ├── model
    │   └── User.java
    └── DemoApplication.java
```

Scanned Package

com.example

↓

All sub-packages are scanned

- ✓ controller
- ✓ service
- ✓ repository
- ✓ model

Example

```
// Main Application Class
@SpringBootApplication
@ComponentScan(basePackages = "com.example")  // scans com.example and sub-packages
public class DemoApplication {
  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }
}
-----------------------------------------------------------------------
// Component Classes
@RestController
public class UserController { }
@Service
public class UserService { }
@Repository
public interface UserRepository extends JpaRepository<User, Long> { }
@Component
public class AppHelper { }
```

Where to Configure

Using @ComponentScan

```
@SpringBootApplication
@ComponentScan(basePackages = "com.example")
public class DemoApplication {
  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }
}
```

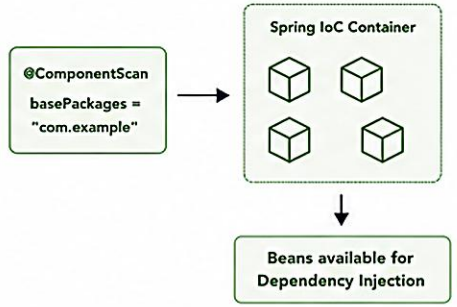
Using @ComponentScan (multiple packages)

```
@ComponentScan(basePackages = {"com.example.service",
                               "com.example.repository"})
public class AppConfig { }
```

Using XML Configuration

```
<context:component-scan base-package="com.example" />
```

Component Scanning in Action



Filters (Optional)

You can include or exclude specific components using filters.

```
@ComponentScan(
  basePackages = "com.example",
  includeFilters = @ComponentScan.Filter(
        type = FilterType.ANNOTATION,
        classes = {MyCustomAnnotation.class}
  ),
  excludeFilters = @ComponentScan.Filter(
        type = FilterType.ASSIGNABLE_TYPE,
        classes = {AbstractClass.class}
  )
)
```

Rules

- Scans only concrete classes (not interfaces or abstract classes).
- Scans the classpath at application startup.
- Default scope of detected beans is singleton.
- If multiple beans of the same type exist, use @Qualifier to resolve ambiguity.

Default Scanning in Spring Boot

Spring Boot automatically configures component scanning for the package of the main application class and its sub-packages. No need to explicitly add @ComponentScan in most cases.

Common Use Cases

- Service layer classes (@Service)
- Repository classes (@Repository)
- REST controllers (@RestController)
- Utility / helper classes (@Component)

Key Takeaway

Component scanning allows Spring to automatically discover and register components, reducing manual configuration and making applications easier to develop and maintain.

@COMPONENT ANNOTATION

The generic stereotype — marks a class as a Spring-managed bean eligible for component scanning.

EmailService.java

```
@Component
public class EmailService {
    public void sendEmail(String to, String body) {
        // send logic
    }
}
```

Annotation	Purpose
@Component	Generic stereotype for any managed component.
@Service	Service layer (business logic).
@Repository	Data access layer (DAO).
@Controller	Web layer (Spring MVC).

BEST PRACTICES

- Use @Component only when no specialized stereotype fits; prefer @Service, @Repository, or @Controller for clarity.
- The registered bean name defaults to the class name in camelCase (e.g. emailService).

KEY TAKEAWAY @Component is the foundation of Spring's stereotypes — @Service, @Repository, and @Controller build on it.

Spring Stereotype Annotations

Core Spring Annotations for Component Scanning

Spring uses stereotype annotations to **automatically detect and register** beans in the Application Context

@Component

Purpose

Generic stereotype for any Spring-managed component

Use When

No other specific stereotype (Service, Repository, Controller) is appropriate

Example

```
@Component
public class EmailClient { }
```

@Service

Purpose

Marks a class as a Service layer component (business logic)

Use When

Creating service classes that contain business logic and use repositories

Example

```
@Service
public class UserService { }
```

@Repository

Purpose

Marks a class as a Repository (data access layer)

Use When

Creating DAO/Repository classes for database operations

Example

```
@Repository
public class UserRepository { }
```

@Controller

Purpose

Marks a class as a Spring MVC Controller (web layer)

Use When

Creating web controllers to handle HTTP requests and responses

Example

```
@Controller
public class UserController { }
```

Key Points

- All are specialized forms of @Component
- Detected via Component Scanning
- Registered as beans in the Application Context
- Improve code readability and maintainability
- Enable role-based organization of layers

@SERVICE ANNOTATION

Marks a class as a service-layer component that contains business logic.

UserService.java

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User findById(Long id) {
        return userRepository.findById(id)
            .orElseThrow();
    }
}
```

BEST PRACTICES

- Keep business logic in @Service classes.
- Make services stateless whenever possible.
- Use constructor injection for dependencies.
- Follow the single responsibility principle.

KEY TAKEAWAY @Service marks the business logic layer, improving readability and enabling clean, layered architecture.

@REPOSITORY ANNOTATION

Marks a class as a data-access-layer (DAO) component with automatic exception translation.

UserRepository.java

```
@Repository
public class UserRepository {
    private final JdbcTemplate jdbcTemplate;

    public UserRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public User findById(Long id) {
        String sql = "SELECT * FROM users WHERE id = ?";
        return jdbcTemplate.queryForObject(sql,
            new Object[]{id}, new UserRowMapper());
    }
}
```

BEST PRACTICES

- Keep only data-access logic in @Repository classes.
- Let Spring translate SQL exceptions automatically.
- Prefer Spring Data JPA repositories when possible.
- Use meaningful names (UserRepository, OrderDao).

KEY TAKEAWAY @Repository enables automatic detection and exception translation for the persistence layer.

@CONTROLLER ANNOTATION

Marks a class as a web-layer component that handles HTTP requests and returns responses.

UserController.java

```
@Controller
@RequestMapping("/user")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id, Model model) {
        model.addAttribute("user", userService.findById(id));
        return "user-details";
    }
}
```

BEST PRACTICES

- Keep controllers thin — business logic belongs in @Service.
- Use meaningful request mappings and method names.
- Validate input and handle exceptions properly.
- Follow RESTful principles for API controllers.

KEY TAKEAWAY @Controller works with Spring MVC to route HTTP requests and delegate to the service layer.

TRY IT YOURSELF — EXERCISE 4: STEREOTYPE ANNOTATION MAP

Reinforces: mapping Northstar CRM types to @Service, @Repository, @RestController, and plain domain — an architecture exercise.

STEREOTYPE MAP (COMPLETE IN notes/stereotype-map.md)

Northstar CRM Type	Stereotype / Note
CustomerService	@Service — business logic layer, orchestrates the repository and notifier.
CustomerRepository (interface)	No annotation — plain contract; @Repository lives on the implementation.
InMemoryCustomerRepository	@Repository — implements CustomerRepository, the DAO layer.
CustomerController	@RestController — exposes HTTP endpoints for the CRM.
Customer (model)	Plain Java — no Spring annotation unless a later module requires one.
NotificationService	@Service — business logic layer, sends customer notifications.

Answer-key skeleton

```
@Service
public class CustomerService { ... }
@Repository
public class InMemoryCustomerRepository
    implements CustomerRepository { ... }
@RestController
public class CustomerController { ... }
```

- **Singleton caution —** default Spring beans are singletons; mutable instance fields on CustomerService are dangerous under concurrent requests.
- **Lab prep —** Lab 22 requires docs/dependency-graph.md naming these exact beans — here you only sketch the names.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-stereotype-map.md.

BUILDING LOOSELY COUPLED SYSTEMS

Depend on abstractions, not concrete implementations — Spring's IoC container and DI make it practical.

TIGHTLY COUPLED

- OrderService depends on the concrete EmailService class.
- Difficult to change or replace EmailService.
- Harder to test — the real EmailService is required.

LOOSELY COUPLED

- OrderService depends on a PaymentService interface.
- Any implementation can be injected at runtime.
- Easier to test, maintain, and extend.

Programming to an Interface

```
public interface PaymentService {
    void pay(double amount);
}

@Service
public class CreditCardPaymentService implements PaymentService {
    public void pay(double amount) { /* charge card */ }
}
```

KEY TAKEAWAY Program to interfaces and let Spring inject the implementation — the key habit behind loose coupling.

SPRING BEAN RELATIONSHIPS

Beans collaborate through Dependency Injection — the container manages these relationships for you.

Relationship	Description
Dependency	One bean requires another to do its work (constructor/setter injection).
Association	Beans are loosely linked and can work independently.
Composition	Strong ownership — the owned bean cannot exist without the owner.
Inheritance	A child bean inherits configuration from a parent bean definition.
References	A bean holds a reference to a collection of other beans.

Dependency via Constructor Injection

```
@Service
public class OrderService {
    private final PaymentService paymentService;
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

KEY TAKEAWAY Dependency Injection and the IoC container manage bean relationships, producing testable applications.

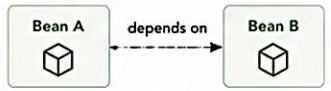
Bean Relationships (Dependencies) in Spring

In Spring, beans rarely work in isolation.
They collaborate with other beans to fulfill their responsibilities.

spring by VMware Tanzu

What is Bean Relationship?

A bean relationship (or dependency) means one bean needs another bean to perform its work.
Spring IoC container creates and wires these beans together.



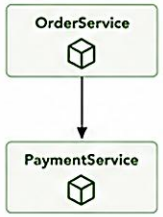
Common Dependency Types

- **Hard Dependency**
  Bean A cannot work without Bean B.
  (Required dependency)
- **Soft Dependency**
  Bean A can work without Bean B, if available it uses it.
  (Optional dependency)
- **Collection Dependency**
  Bean A depends on a collection of Bean B.
- **Self Dependency**
  Bean depends on itself.
  (Rare, use with caution)

Types of Bean Relationships

1. One-to-One

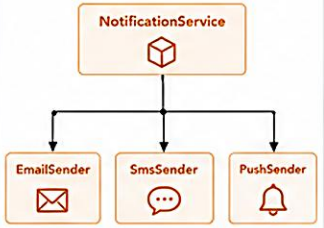
One bean depends on another single bean.



Example:
OrderService depends on PaymentService.

2. One-to-Many

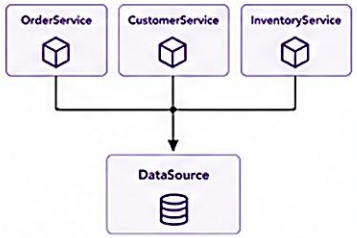
One bean depends on multiple beans.



Example:
NotificationService uses multiple sender implementations.

3. Many-to-One

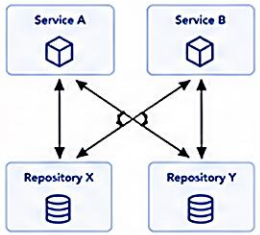
Multiple beans depend on one common bean.



Example:
Many services use the same DataSource.

4. Many-to-Many

Multiple beans depend on multiple beans.



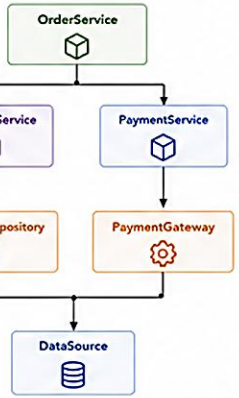
Example:
Services share multiple repositories and vice versa.

How Spring Manages Relationships

1. **Scanning**
   Spring scans components (@Component, @Service, etc.) and registers beans.
2. **Dependency Resolution**
   Container analyzes constructor, setter, or field dependencies.
3. **Injection**
   Spring injects the required bean(s) into dependent beans.
4. **Ready to Use**
   All beans are wired and ready for use.

Example : E-Commerce Application

- **OrderService**
  Depends on CustomerService and PaymentService.
- **CustomerService**
  Depends on CustomerRepository.
- **PaymentService**
  Depends on PaymentGateway.
- **CustomerRepository**
  Depends on DataSource.



Dependency Injection in Action

Constructor Injection (Recommended)

```
@Service
public class OrderService {
   private final CustomerService customerService;
   private final PaymentService paymentService;

   @Autowired
   public OrderService(CustomerService customerService,
                       PaymentService paymentService) {
      this.customerService = customerService;
      this.paymentService = paymentService;
   }
}
```

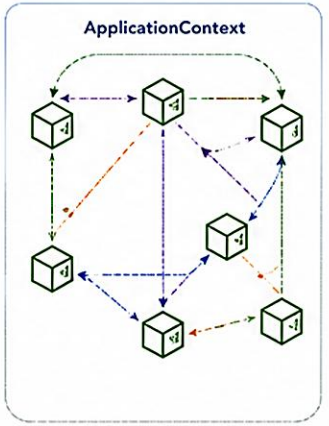
Setter Injection

```
@Service
public class OrderService {
  private CustomerService customerService;

  @Autowired
  public void setCustomerService(CustomerService customerService) {
     this.customerService = customerService;
  }
}
```

Field Injection (Not Recommended)

```
@Service
public class OrderService {
     @Autowired
     private CustomerService customerService;
}
```

Bean Relationship in ApplicationContext



Legend

- One-to-One
- One-to-Many
- Many-to-One
- Many-to-Many

Benefits

- Loose Coupling
- Easy Testing
- Modular Design
- Easier Maintenance
- Reusability

Key Points

- Spring promotes loose coupling through Dependency Injection.
- Relationships can be one-to-one, one-to-many, many-to-one, or many-to-many.
- Constructor injection is the recommended best practice.
- Spring IoC container manages the lifecycle and dependencies.
- Well-defined relationships lead to clean, testable, and maintainable code.

TRY IT YOURSELF — EXERCISE 5: BEAN GRAPH SKELETON (STARTER)

Reinforces: constructor DI wiring in plain Java, no Spring runtime yet — fill in every blank before compiling.

IocDemo.java + collaborators (STARTER — fill in every ____)

```
interface CustomerRepository {
    String findName(String id);
}
class InMemoryCustomerRepository implements CustomerRepository {
    public String findName(String id) {
        // TODO: return "Amina Khan" for "CUS-1001", else "UNKNOWN"
        if ("CUS-1001".equals(id)) return _____;
        return "UNKNOWN";
    }
}
class CustomerService {
    private final CustomerRepository _____;  // TODO: field name repo
    CustomerService(CustomerRepository repo) {
        this._____ = _____;  // TODO: assign
    }
}
```

#	What To Do
1	Create CustomerRepository.java, InMemoryCustomerRepository.java, CustomerService.java, IocDemo.java under mini-src/com/northstar/crm/.
2	Fill every ____ / TODO shown at left — blanks are not valid Java and will not compile.
3	Compile: javac -d mini-out mini-src/com/northstar/crm/*.java; run: java -cp mini-out com.northstar.crm.IocDemo
4	Expected output: CUS-1001 | Amina Khan. In notes/bean-graph-sketch.md, draw IocDemo → CustomerService → CustomerRepository.

KEY TAKEAWAY TRY IT NOW — This is a STARTER, not a solution: complete every blank in exercises/exercise-05-bean-graph-skeleton.md, then compile and run before continuing.

ENTERPRISE APPLICATION STRUCTURE

A clean, layered package structure keeps enterprise Spring applications maintainable and easy to navigate.

1. **Presentation**
2. **Business**
3. **Data Access**
4. **Database**
5. **External Systems**

Package Structure Example

```
com.example.bank
|-- controller      REST Controllers
|-- service         Business Services
|-- repository      Data Access Interfaces
|-- entity          JPA Entities
|-- dto             Request / Response DTOs
`-- config          Configuration classes
```

BEST PRACTICES

- Keep controllers thin; delegate to services.
- Use interfaces for services and repositories.
- Leverage DI for loose coupling between layers.
- Handle exceptions consistently across layers.

KEY TAKEAWAY A clear layered structure with IoC-based wiring produces flexible, testable, future-ready systems.

Enterprise Application Structure

A well-structured enterprise application is modular, scalable, maintainable and secure.
It separates concerns into layers and follows standard best practices.

spring by VMware Tanzu

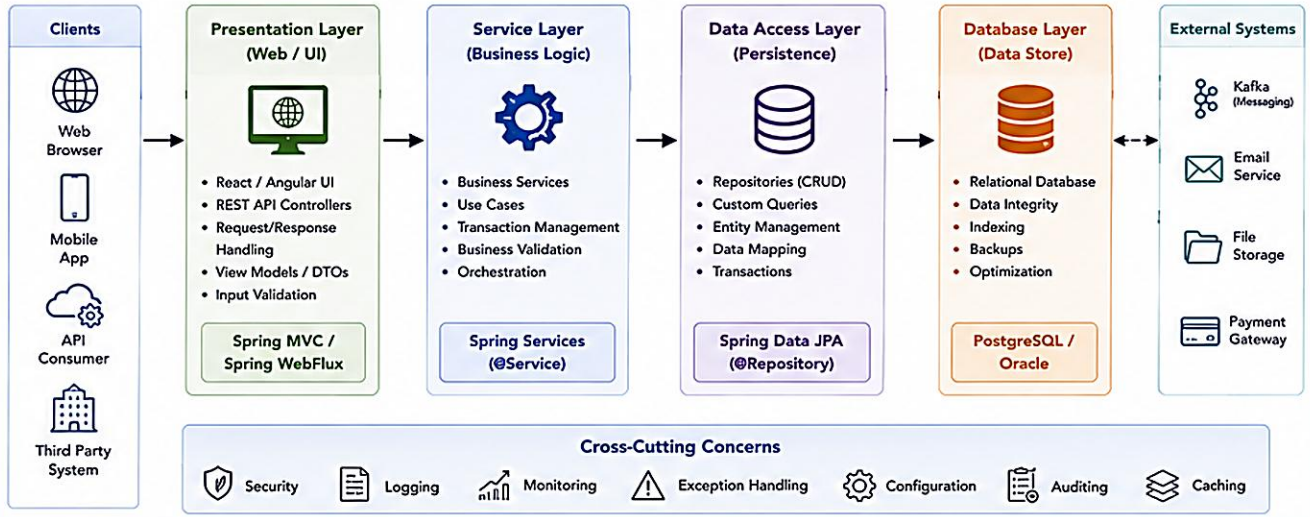
Core Principles

- **Separation of Concerns** — Each layer has a specific responsibility.
- **Modularity** — Components are loosely coupled and highly cohesive.
- **Scalability** — Designed to scale horizontally and vertically.
- **Maintainability** — Easy to test, update and extend.
- **Security** — Security is integrated at every layer.

Typical Technology Stack

Frontend	:	React / Angular
Backend	:	Spring Boot
Security	:	Spring Security, JWT
Persistence	:	Spring Data JPA
Database	:	PostgreSQL / Oracle
Messaging	:	Apache Kafka
Build	:	Maven / Gradle
Deploy	:	Docker, Kubernetes

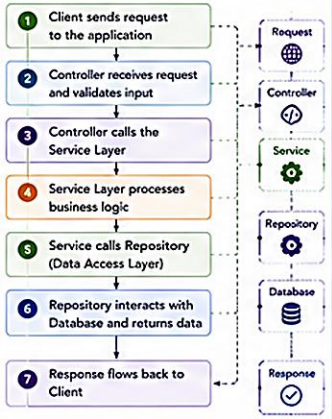
Typical Layered Enterprise Application



Example Project Structure (Maven / Gradle)

```
enterprise-app
└ src
  └ main
    └ java
      └ com.example.enterpriseapp
        ├ controller      # Presentation Layer
        ├ service         # Service Layer
        ├ repository      # Data Access Layer
        ├ model           # Entity / Domain
        ├ dto             # Data Transfer Objects
        ├ config          # Configuration Classes
        ├ exception       # Exception Classes
        └ util            # Utility Classes
    └ resources
      ├ application.yml   # Configuration
      ├ static/           # Static Files
      └ templates/        # Templates (if any)
  └ test
    └ java ...
```

Request Flow



Key Architectural Patterns

Layered Architecture	Organizes the application into layers with well-defined responsibilities.
Dependency Injection	Spring IoC container manages dependencies between components.
Repository Pattern	Abstracts data access logic and provides a clean API.
DTO Pattern	Used to transfer data between layers and over the network.
Service Layer Pattern	Encapsulates business logic and maintains transaction boundaries.
Configuration Pattern	Externalized configuration for different environments.

Key Benefits

- Clear structure and separation of layers
- Easier testing and debugging
- High maintainability and extensibility
- Independent scalability of layers
- Reusability of components
- Supports Agile and DevOps practices

Cross-Cutting Concerns

- **Security** : Authentication, Authorization, Validation
- **Logging** : Centralized logging with SLF4J / Logback
- **Monitoring** : Actuator, Prometheus, Grafana
- **Exception Handling** : Global exception handling
- **Configuration** : Externalized configuration (properties/ YAML)

Key Takeaways

- A clear structure is the foundation of a successful enterprise application.
- Use layers to separate responsibilities and improve maintainability.
- Follow standard patterns and best practices.
- Leverage Spring ecosystem for rapid and robust development.
- Design for scalability, security and observability from the start.

A strong structure today ensures a reliable, scalable and maintainable application tomorrow.

Well-defined layers | Loose coupling | High cohesion | Testability | Scalability | Security | Maintainability

TRY IT YOURSELF — EXERCISE 6: LAB 22 READINESS CHECKLIST

Capstone pre-lab check — confirm tools, paths, and scope boundaries before opening Lab 22.

READINESS CHECKLIST (RECORD IN notes/lab22-readiness.md)

Check	What To Confirm
Tool Check	Run java -version and mvn -version; record JDK 21 and Maven 3.9+.
Copy Target	Windows: %USERPROFILE%\java-bootcamp\examples\lab22-crm — macOS: ~/java-bootcamp/examples/lab22-crm
Deliverables Preview	List 4 things NOT finished in pre-lab: stereotype beans, constructor DI, dependency-graph.md, Spring tests.
Evidence Folder	Create a notes/screenshots/lab-22/ placeholder for later lab screenshots.

Setup (PowerShell) — from EXERCISES-INDEX.md

```
cd $env:USERPROFILE\java-bootcamp
New-Item -ItemType Directory -Force -Path examples\module-22-exercises | Out-Null
cd examples\module-22-exercises
java -version
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before opening Lab 22: exercises/exercise-06-lab22-readiness.md.

LAB OVERVIEW — SPRING IoC AND DEPENDENCY INJECTION

Lab 22: Spring IoC and Dependency Injection — Northstar CRM Bean Graph

BUSINESS SCENARIO

- The Northstar CRM stores customer identity, contact details, lifecycle status, and accounts.
- Leadership freeze: Spring owns the object graph; constructor injection is preferred; every component carries a stereotype; the dependency graph is documented.
- Manual new InMemoryCustomerRepository() calls inside services block swapping persistence, metrics, and notifiers for tests and production.

LEARNING OBJECTIVES

- Explain IoC vs. dependency lookup — CRM services should never call new on collaborators.
- Declare @Component, @Service, and @Repository beans for the customer domain.
- Prefer constructor injection for CustomerService's dependencies.
- Observe bean lifecycle via @PostConstruct / @PreDestroy.
- Document and export a dependency graph for review.

WHAT YOU BUILD

- Copy lab21-crm to lab22-crm; declare @Repository/@Service beans; refactor CustomerService to constructor DI; wire @RestController; add @PostConstruct/@PreDestroy; prove unit + @SpringBootTest.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) with correlation lab-request-001 anchor every example.

LAB TASKS AND DELIVERABLES

Northstar CRM Bean Graph — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch Lab 21 and confirm the Spring Boot scaffold starts cleanly.
2	Model the Customer domain type without Spring annotations.
3	Declare @Repository / @Service beans for repository and notifier collaborators.
4	Refactor CustomerService to constructor injection with final fields.
5	Wire CustomerController as a Spring MVC bean via constructor injection.
6	Add @PostConstruct / @PreDestroy lifecycle logging to CustomerService.
7	Prove testability — pure unit test plus a @SpringBootTest integration test.
8	Document docs/dependency-graph.md and run the failure experiments.

EXPECTED DELIVERABLES

- CustomerService, CustomerRepository, NotificationService as constructor-injected beans.
- Lifecycle evidence (@PostConstruct/@PreDestroy).
- Unit test with fakes + a @SpringBootTest IT.
- docs/dependency-graph.md matching the real constructors.
- Evidence for CUS-1001 / CUS-1002 with correlation lab-request-001.

RUBRIC (100 MARKS)

- Core implementation 30 · Integration 15 · Failure handling 15 · Tests 10 · Security 10 · Docs 10 · Env 10

KEY TAKEAWAY Field @Autowired as the primary pattern, or any new of a Spring-managed collaborator, is an automatic deduction.

MODULE 22 SUMMARY

Spring Core and Inversion of Control — what we covered and how it fits together.

KEY CONCEPTS COVERED

IoC Container

Creates, configures, and manages beans and their lifecycle.

Dependency Injection

Constructor, setter, and field injection wire collaborators.

Bean Lifecycle & Scope

Init/destroy callbacks; singleton and prototype scopes.

Configuration Options

XML, Java config, or annotations define beans.

Loose Coupling

Program to interfaces; let Spring supply the implementation.

Stereotypes

@Component, @Service, @Repository, @Controller drive scanning.

MODULE FLOW RECAP

1. **IoC Container**
2. **Dependency Injection**
3. **Bean Lifecycle & Scope**
4. **Component Scanning**
5. **Loosely Coupled Design**

KEY TAKEAWAY Spring IoC and Dependency Injection enable loose coupling and better design — the foundation for everything ahead.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Spring IoC and dependency injection.

1. What does the Spring IoC container primarily manage?

A. Database connections
B. Bean creation, configuration, and lifecycle
C. HTTP sessions only
D. File I/O operations

2. Which injection type does the Spring team recommend for mandatory dependencies?

A. Field injection
B. Setter injection
C. Constructor injection
D. Static injection

3. What is the default scope of a Spring bean?

A. Prototype
B. Request
C. Session
D. Singleton

4. Which annotation marks a data-access-layer (DAO) component?

A. @Controller
B. @Service
C. @Repository
D. @Configuration

KEY TAKEAWAY IoC and DI are the foundation of every Spring-managed enterprise application.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on beans, scopes, and component scanning.

5. Why is field injection discouraged in production code?

A. It is slower at runtime

B. It hides dependencies and is hard to test without Spring

C. It cannot use interfaces

D. It requires XML configuration

6. Which lifecycle annotation runs cleanup logic before a bean is destroyed?

A. @PostConstruct

B. @PreDestroy

C. @Bean

D. @Scope

7. A new instance is created every time a bean is requested under which scope?

A. Singleton

B. Prototype

C. Application

D. Request

8. What lets Spring automatically detect @Service and @Repository classes?

A. Manual XML bean registration

B. Component scanning

C. The JVM classloader alone

D. Reflection API alone

KEY TAKEAWAY Component scanning plus stereotype annotations let Spring find and wire your beans with almost no manual config.

"The framework calls your code — not the other way around."

MODULE 22 COMPLETE

Spring Core and Inversion of Control

You can now build Spring beans, apply constructor/setter/field injection, manage bean lifecycle and scope, and design loosely coupled enterprise applications.